ChatGPT

# Automation using n8n / Zapier

**Document type:** College-seminar-ready, printable (PDF-friendly) documentation with labs, diagrams, comparisons, and speaker notes
**Target audience:** Undergraduate students with basic programming + web knowledge (HTTP, JSON, APIs, auth basics)
**Edition date:** 2026-02-15 (Asia/Kolkata)
**Estimated final length target:** ~40 pages, ~16,800 words (range 16,000–18,000)
**Layout assumption:** US Letter or A4; ~11–12pt body text; 1.0–1.15 line spacing; figures/tables interleaved; code in monospace.

## Executive summary

Workflow automation is the engineering practice of encoding repeatable work into reliable, observable systems that move data and trigger actions across tools, services, and APIs. A widely used formal definition (from the Workflow Management Coalition [1]) describes workflow as "the automation of a business process, in whole or part," where information or tasks pass among participants according to procedural rules. [2] This definition maps cleanly to modern "no-code/low-code" automation platforms: triggers detect events, actions perform work, and rules govern routing, timing, and error handling.

This document compares two popular approaches: - **n8n**: a source-available automation platform intended for technical teams, blending a visual editor with code (JavaScript/Python), and emphasizing controllable deployments (cloud or self-hosted). n8n markets "over 500 integrations" and "500+ integrations" in its current product messaging, while its repository describes "400+ integrations"—a reminder that "integration counts" vary by what's included (built-in nodes vs. community nodes vs. generic HTTP APIs). [3] - **Zapier**: a managed SaaS automation platform centered on ease-of-use, broad connectivity (8,000+ integrations), and operational simplicity for individuals and organizations. [4]

A key technical difference is **how complexity is priced and governed**: - n8n cloud plans charge by **workflow executions** (a full run of a workflow), explicitly stating that a single execution covers the entire workflow "regardless of complexity" and "how many steps" it contains. [5] - Zapier charges by **tasks**, counted per **successful action**; triggers do not count, and many built-in "utility" steps (e.g., Filter, Formatter, Paths) are explicitly non-task-counting. [6]

From an instructional perspective, both tools teach core automation thinking: - **Modeling**: define inputs/outputs, data schemas, and invariants. - **Control flow**: filters, branches, loops, schedules, and webhooks. - **Reliability**: idempotency, retries, rate-limits, and compensations (undo/rollback). - **Security/privacy**: least privilege, secret handling, logging, and compliance constraints.

**Practical recommendation (for undergraduate seminar contexts):** - Teach **Zapier first** to establish automation fundamentals quickly (fast feedback, fewer deployment variables), then teach **n8n** to introduce deeper systems concerns (self-hosting tradeoffs, queue-based scaling, credential encryption keys, reverse

proxies, and more explicit API work). This sequencing mirrors how the platforms themselves position their strengths: Zapier as managed agility; n8n as technical control. [7]

**What you will be able to do after this seminar:** design and implement six complete workflows (three per platform), diagnose common failures, estimate cost drivers (executions vs. tasks), and apply a security/privacy checklist aligned with mainstream frameworks (NIST CSF + NIST Privacy Framework + OWASP API risk guidance). [8]

### Speaker notes

Emphasize that "automation" is not merely drag-and-drop. It is: (a) requirements (what outcome), (b) contracts (what data), (c) correctness (what must always be true), (d) reliability (what happens when it breaks), and (e) governance (who can change it and how we audit changes). Tie this directly to how both platforms expose history, logging, and access controls. [9]

## Document map and indexing

This section provides a PDF-ready indexing package: page plan, Table of Contents, List of Figures/Tables/Code, and a term index.

### Word count and page breakdown by section

> **Target**: ~16,800 words across 40 pages (≈420 words/page average). Actual pagination depends on font, spacing, and how many screenshots you embed.

| Section | Planned pages | Target words |
|---|---|---|
| Front matter (Title, executive summary, how to use) | 1–3 | 1,000 |
| Indexing (TOC, lists, term index) | 4–6 | 1,000 |
| Foundations of automation | 7–12 | 2,600 |
| Platform deep dives (n8n + Zapier) | 13–18 | 2,800 |
| Hands-on labs: n8n (3 workflows) | 19–25 | 3,000 |
| Hands-on labs: Zapier (3 workflows) | 26–32 | 3,000 |
| Comparative analysis | 33–36 | 1,600 |
| Best practices + troubleshooting | 37–39 | 1,400 |
| Use cases + ethics + future trends (capstone) | 40 | 400 |
| **Total** | **40** | **16,800** |

## Page-by-page plan

| Page | Contents | Target words |
|---|---|---|
| 1 | Title page, document control, abstract | 250 |
| 2 | Executive summary (part 1) | 450 |
| 3 | Executive summary (part 2), how to use this document | 300 |
| 4 | Table of Contents | 350 |
| 5 | List of Figures, Tables, Code Listings + screenshot requests | 350 |
| 6 | Index of terms | 300 |
| 7 | Foundations: workflow definition, trigger/action mental model | 450 |
| 8 | Foundations: data contracts, schemas, JSON discipline | 420 |
| 9 | Foundations: control flow (filters/branches), state and idempotency | 420 |
| 10 | Foundations: errors as first-class (exceptions, retries, compensation) | 420 |
| 11 | Foundations: observability (logs, run history, audit logs) | 420 |
| 12 | Foundations: security/privacy baseline | 470 |
| 13 | n8n overview: philosophy, editor, data model | 420 |
| 14 | n8n architecture + hosting + scaling (queue mode) | 450 |
| 15 | n8n pricing + governance + licensing | 450 |
| 16 | Zapier overview: Zaps, tasks, built-in tools, limits | 450 |
| 17 | Zapier security/compliance + admin/audit | 420 |
| 18 | Cross-platform design patterns (webhooks, polling, ETL, RPA boundaries) | 530 |
| 19 | n8n Lab A (beginner) | 450 |
| 20 | n8n Lab A testing + extensions + diagram | 420 |
| 21 | n8n Lab B (intermediate) | 450 |
| 22 | n8n Lab B reliability + rate limits + diagram | 420 |
| 23 | n8n Lab C (advanced) | 500 |
| 24 | n8n Lab C scaling + error workflows + diagram | 420 |
| 25 | n8n lab recap + troubleshooting mini-cases | 340 |
| 26 | Zapier Lab A (beginner) | 450 |
| 27 | Zapier Lab A task accounting + diagram | 420 |

| Page | Contents | Target words |
|---|---|---|
| 28 | Zapier Lab B (intermediate) | 450 |
| 29 | Zapier Lab B filters/formatter + diagram | 420 |
| 30 | Zapier Lab C (advanced) | 500 |
| 31 | Zapier Lab C Paths + error handling + diagram | 420 |
| 32 | Zapier lab recap + troubleshooting mini-cases | 340 |
| 33 | Comparison: capability matrix + integration ecosystems | 450 |
| 34 | Comparison: cost modeling (executions vs tasks) | 420 |
| 35 | Comparison: extensibility + governance + security tradeoffs | 420 |
| 36 | Decision framework: choose by constraints, not preferences | 310 |
| 37 | Best practices: design, testing, deployments | 450 |
| 38 | Best practices: secrets, least privilege, privacy-by-design | 420 |
| 39 | Troubleshooting: common failures and playbooks | 530 |
| 40 | Use cases + ethics + future trends + capstone prompt | 400 |

## Table of Contents



## List of Figures

## List of Tables



## List of Code Listings



## Screenshot requests for final layout

Because screenshots should be pulled from official documentation during final PDF layout (for consistency and licensing clarity), include placeholders and capture them from: - n8n Webhook node: Test vs Production URLs and “Respond” options. 18
- n8n reverse proxy webhook URL configuration (WEBHOOK_URL, N8N_PROXY_HOPS). 19
- Zapier editor showing a Filter step and warning behavior. 20
- Zapier history page and run details view. 21
- Zapier Paths editor with branches and limitation note. 22

## Index of terms



## Speaker notes

Tell students that indexing is not formatting “decoration.” Indexes and lists are operational tools: they enable faster navigation, quicker debugging, and reproducibility when teams share workflows. Link this to audit logs and run histories as another “indexing layer,” but for runtime behavior. 30

# Foundations of automation

## What “workflow automation” means in rigorous terms

A workflow is best understood as a **controlled, repeatable execution** of tasks guided by explicit rules. The Workflow Management Coalition [1] definition emphasizes that workflows automate business processes fully or partially and route information/tasks among participants under procedural rules. [2] This definition highlights four properties that are directly useful when building automations in n8n or Zapier:

- **A workflow has structure** (a graph of steps, not a loose script).
- **A workflow moves artifacts** (data, files, messages).
- **A workflow coordinates actors** (humans and software services).
- **A workflow enforces rules** (conditions, approvals, timing).

Modern automation platforms implement these as event-driven DAGs (directed acyclic graphs) most of the time, with controlled “loop-like” constructs (batching, pagination, replay, fan-out) added as patterns. The research tradition around workflow patterns formalizes these capabilities and uses them to compare workflow systems. [31]

## A shared mental model for both platforms

Think in a standard pipeline:

```
flowchart LR
  E[Event source<br/>app/API/human] --> T[Trigger<br/>detect event]
  T --> V[Validate + normalize<br/>schema, required fields]
  V --> R{Rules<br/>filter/branch}
  R -->|Path 1| A1[Action(s)]
  R -->|Path 2| A2[Action(s)]
  A1 --> O[Outputs<br/>records, messages, files]
  A2 --> O
  O --> H[History + logs<br/>run record, audit]
```

This diagram corresponds to: - Zapier: Trigger → steps (Filter/Formatter/Paths/actions) → Zap history. [32]
- n8n: Trigger nodes (Webhook, Scheduler, etc.) → nodes for mapping/transform → execution logs and (optionally) error workflows. [33]

## Vocabulary mapping

| Concept | n8n term | Zapier term | Why it matters |
|---|---|---|---|
| Workflow | workflow | Zap | Unit of automation ownership + change control [34] |

| Concept | n8n term | Zapier term | Why it matters |
|---|---|---|---|
| Start condition | trigger node | trigger | Defines event semantics (poll vs webhook) [29] |
| Step | node | step/ action | Where data is transformed and work occurs |
| Cost metric | execution | task | Drives design tradeoffs (more steps vs more runs) [11] |
| Branching | IF-like nodes | Paths | Affects complexity limits (Zap step cap) [35] |
| Run logs | executions tab/ logs | Zap history | Debugging and compliance evidence [36] |

## Data contracts and JSON discipline

Automation fails most often at **interfaces**, not in the middle of a workflow. For undergraduates, the single most useful professional habit is to define a stable "internal schema" for your workflow early and normalize incoming payloads to that schema.

n8n makes this explicit through its internal item model: data items are typically wrapped as objects with a `json` key; documentation teaches returning arrays of `{ json: {...} }` and notes that newer versions can auto-add `json` if missing. [15] This design choice encourages structured thinking: *every step should know what fields exist and what types they are*.

Zapier's UI encourages a similar practice but in a different way: mapping fields between steps, applying Formatter transforms, and using Filter conditions to guard later steps. [37]

## Reliability is not optional: exceptions, retries, and compensations

Workflow research treats exceptions as first-class because deviations during execution are normal (timeouts, invalid inputs, missing permissions, rate limits). A pattern-based view helps you reason about the "shape" of failure handling: detect → classify → respond → continue/abort. [38]

In practice: - **Retry** addresses transient failures (network hiccups, momentary API overload). - **Backoff + batching** addresses rate limiting (e.g., HTTP 429). - **Compensation** addresses partial success (step 1 succeeded, step 2 failed; you may need to undo step 1).

n8n documents built-in strategies like batching and "Retry on Fail" to mitigate rate-limit failures for API calls. [23] Zapier enforces operational guardrails by automatically turning off Zaps with a high error ratio (95% errors in the last 7 days), forcing investigation rather than allowing silent failure. [39]

### A minimal idempotency model

Most automations are “at least once” in practice: events might be delivered twice, or you might replay a run, or a webhook sender retries. In HTTP semantics, an idempotent method is one where multiple identical requests have the same intended effect as one; PUT and DELETE (and safe methods) are idempotent by definition. [14]

For non-idempotent operations (often POST-like “create”), implement an **idempotency key** strategy:

```
idempotency_key = hash(event_source + event_id + action_type)
if already_processed(idempotency_key):
    skip
else:
    do_action()
    mark_processed(idempotency_key)
```

In Zapier, “replay” and re-running can cause repeated actions and repeated task consumption; Zapier’s definition of what counts toward tasks includes rerunning previously successful steps when you replay an entire Zap run. [40] In n8n, a “workflow execution” runs from start to finish, and if retried or re-triggered, you can similarly duplicate side effects unless you guard them. [41]

### Speaker notes

Spend time on “data contracts” and “idempotency.” Students often think failures are rare and duplicates won’t happen. Show them that duplicates are common because systems retry. Use the RFC definition to legitimize why we care about retries and what “idempotent” means in engineering, not just theory. [14]

## Platform deep dives

### n8n overview

n8n positions itself as a tool for “technical teams” combining visual workflows with code, including the ability to write JavaScript/Python and use packages, and it advertises “over 500 integrations.” [42]

#### Architecture and core components

At a conceptual level, n8n consists of: - A **workflow editor** (graphical canvas for nodes). - An **execution engine** that runs nodes and passes items downstream. - An **integrations layer** (built-in nodes + community nodes + generic HTTP). - An **execution store** (logs/history, and for self-hosting, the chosen DB/storage). - Optional scaling mode: **queue mode** with workers.

n8n’s webhook model is a good “systems” teaching tool because it explicitly distinguishes **test** vs **production** webhook URLs and how registration works. The Webhook node documentation explains that in test mode, a webhook is registered when you listen/execute; in production, registration happens when you publish; production runs don’t display data in the editor but are visible via execution history. [18]

**Security and hosting implications**

n8n explicitly warns that self-hosting requires expertise and that mistakes can lead to data loss, security issues, and downtime—so the "platform choice" is also a "who operates it" choice. [43]

For hosted n8n Cloud, the security page describes encryption at rest for stored tokens/credentials on encrypted disks (Azure server-side encryption, AES-256, FIPS 140-2 compliant implementation), TLS in transit, SSL certificate management via Cloudflare [44], and operational defenses like WAF and intrusion detection. [45]

For self-hosted n8n, the same security page places responsibility on operators to implement TLS in front of n8n and to manage encryption at rest (e.g., encrypted partitions / disk encryption). [45] A practical lesson: **with self-hosting, compliance becomes your engineering problem**—not just a checkbox.

**Pricing model and what "execution" means**

n8n's pricing page states that plans include unlimited users and workflows and "every integration," with pricing based on monthly workflow executions. [46] It defines an "execution" as a **single run of your entire workflow**, explicitly stating that step count and data volume do not change the fact it is one execution. [47]

As of this edition date, the pricing page shows: - Starter: 20€/month billed annually, 2.5K executions, 5 concurrent executions, unlimited users. [48] - Pro: 50€/month billed annually, 10K executions, 20 concurrent executions, global variables and admin roles listed. [46] - Business: 667€/month billed annually, 40K executions, self-hosted, includes SSO (SAML/LDAP), environments, scaling options, Git-based version control. [49]

These details matter for design: if you can combine multiple "small actions" into one workflow execution (e.g., batch processing), n8n's pricing model incentivizes building richer workflows per run, unlike per-step accounting. [47]

**Scaling with queue mode**

Queue mode decouples request intake (main) from execution (workers). n8n's docs describe multi-main requirements: mains in queue mode connected to Postgres + Redis, mains and workers on the same version, multi-main toggle enabled, and sticky sessions behind a load balancer. [10]

```
flowchart TB
  subgraph ClientSide[Clients & Event Sources]
    S1[Webhook sender]
    S2[Scheduler / cron]
    S3[Polling triggers]
  end

  subgraph Main[n8n main processes]
    UI[Editor/UI + API]
    WH[Webhook handler]
```

```
  ENQ[Enqueue jobs]
end

subgraph Infra[Shared infrastructure]
  PG[(Postgres<br/>workflows, creds, history)]
  RQ[(Redis<br/>job queue)]
end

subgraph Workers[n8n worker processes]
  WK1[Worker]
  WK2[Worker]
  WK3[Worker]
end

S1 --> WH --> ENQ --> RQ
S2 --> UI --> ENQ
S3 --> UI --> ENQ
RQ --> WK1 --> PG
RQ --> WK2 --> PG
RQ --> WK3 --> PG
UI --> PG
```

This architecture is pedagogically powerful: it illustrates why **stateless scaling** often requires shared databases, queues, version alignment, and load balancing. [10]

#### Credential encryption key as an operational concept

n8n generates an encryption key on first launch and stores it, using it to encrypt credentials before saving them to the database; in queue mode, you must specify the encryption key environment variable for all workers. [24] This is a real-world example of why "secrets management" is not optional: losing keys can break decryption, and inconsistent keys across components can cause failures.

#### Licensing as a governance constraint

n8n documents its "Sustainable Use License" as a fair-code license created by n8n (2022), allowing use/modification/redistribution with limitations—especially restrictions against offering n8n as a paid hosted service or white-labeling it as the core value. [50] n8n also states it does not call itself open source under the Open Source Initiative [51] definition because OSI licenses cannot include certain usage restrictions. [50]

### Zapier overview

Zapier is positioned as an automation platform with extremely broad connectivity. Its Help Center states Zapier offers "over almost 8,000 apps," and its public site describes 8,000+ integrations as part of its ecosystem. [52]

**Core building blocks: Zaps, triggers, actions, tasks**

Zapier's pricing page provides unambiguous definitions: - Zap = automated workflow with a trigger and one or more actions. - Task = counted each time a Zap successfully moves data or completes an action; triggers don't count; polling checks don't count. [53]

The "How is task usage measured" article adds detail: only successful actions count; successful steps inside error handler paths count; replaying entire Zap runs may re-consume tasks for previously successful steps. [54]

A high-value insight for cost-awareness: Zapier explicitly lists built-in steps that do **not** count as tasks, including Filter, Formatter, Path, Delay, Looping, Sub-Zap, Digest, and others. [53] This means Zapier encourages rich logic **inside** the tool without penalizing cost per logic step—cost is driven mainly by external side-effect actions.

**Limits: steps and branching**

Zapier caps Zaps at 100 steps including steps within Paths. [55] Paths have constraints: up to 10 branches per path group, up to 3 nested Path steps, and Paths must be the final step in a Zap. [22]

These constraints are central to pedagogy: students learn that platform limits shape architecture. A common professional response is decomposition: break one huge automation into multiple Zaps and chain them via webhooks or shared tables/storage.

**Webhooks and Code steps**

Zapier supports webhooks as triggers (Catch Hook / Catch Raw Hook). Catch Raw Hook returns unparsed body (max 2MB) plus headers; Catch Hook parses body; Zapier provides the URL you POST to, and notes that you need basic HTTP/API knowledge; Zapier can throttle under load. [56]

Zapier also supports embedded code steps. Its documentation is explicit: - Code steps can run snippets in a constrained environment. - The JavaScript environment runs on Node.js 18. - Code actions use the UTC time zone. - Data from prior steps is accessed through a curated `inputData` mapping (you define key/value mappings in the UI). [17]

These constraints are important. For example, needing explicit `inputData` mapping prevents unconstrained access to all step data, which is helpful for clarity and for limiting accidental data exposure.

**Observability: Zap history and retention limits**

Zapier's Zap history provides run logs and task usage. Zapier states it can only guarantee a maximum of 60 days of run data and will display up to 10,000 runs; exporting is recommended for longer retention. [21]

Zapier's troubleshooting documentation also clarifies an operational safety mechanism: a Zap will automatically turn off if 95% of runs error in the last 7 days (with additional plan-dependent notification/grace behaviors). [39]

#### Security and compliance posture

Zapier's security/compliance pages state that Zapier uses TLS (e.g., TLS 1.2) for web communications and AES-256 for encryption at rest, and claims compliance with GDPR, SOC 2 Type II, and CCPA. [57] Zapier's help center also states it has SOC 2 Type II and SOC 3 certifications. [58]

For governance, Zapier documents an audit log that allows monitoring of account activity, including who performed actions and when (including "Zapier System" as actor). [59]

### Speaker notes

Frame the platform deep-dive as "tradeoffs you can justify": - Zapier optimizes time-to-value and managed operations but imposes architectural constraints (step limits, paths placement). [60]
- n8n enables deeper control and more explicit engineering patterns (queue mode, encryption keys, reverse proxy configuration), which also increases operational burden and risk if the team lacks experience. [61]

## Hands-on labs

These labs are designed to be taught in a seminar format. Each lab includes: objective, prerequisites, "build" steps, testing strategy, and a mini troubleshooting checklist. The workflows are intentionally chosen to represent common automation archetypes: **notify**, **sync/ETL**, and **triage/orchestrate**.

### Labs for n8n

#### Lab n8n A: Event intake and notification via webhook

**Level:** Beginner
**Archetype:** Webhook → validate → notify → return response

**Objective:** Build a simple webhook endpoint that receives JSON, validates required fields, posts a notification to a chosen destination (e.g., email/chat/HTTP endpoint), and returns a structured response.

**Why this lab matters:** n8n's Webhook node teaches production vs test URLs, response behaviors, payload limits, and basic auth choices. [18]

#### Workflow overview diagram

```
flowchart LR
  W[Webhook trigger<br/>POST /lead] --> V[Validate fields<br/>required keys]
  V --> N[Notify downstream<br/>email/slack/http]
  N --> R[Respond to Webhook<br/>200 + receipt]
```

#### Step-by-step build

1) Create a new workflow and add a **Webhook** trigger node. Configure: - Method: POST - Path: `/lead` (use a stable path for demo repeatability) - Authentication: start with "None" for classroom testing; note that the Webhook node supports Basic, Header, JWT, or none. [18]

2) Use the **Test URL** initially. - Start "Listen for test event" and send a sample request. n8n explains that "test" registers a test webhook while listening/executing; production registers when you publish. [18]

3) Add a validation step (choose one of two patterns): - **No-code**: use an "IF" or filter-like node (course-dependent). - **Code node**: validate presence of keys and return a normalized structure.

A minimal code normalization pattern that matches n8n's item model:

```
// n8n Code node: normalize incoming webhook body
const body = $input.first().json.body ?? $input.first().json;

function requireField(obj, key) {
  if (obj[key] === undefined || obj[key] === null || obj[key] === '') {
    throw new Error(`Missing required field: ${key}`);
  }
}

requireField(body, 'email');
requireField(body, 'name');

return [
  {
    json: {
      name: body.name,
      email: body.email,
      source: body.source ?? 'unknown',
      receivedAt: new Date().toISOString(),
    },
  },
];
```

This aligns with n8n's documented expectation that returned items are arrays of objects containing a `json` key. [62]

4) Add a "notification" step. - If you have a native node (e.g., Slack/email) use it. - Otherwise, demonstrate the **HTTP Request** node to call a generic webhook endpoint, emphasizing that n8n's HTTP Request node is designed to access any REST API and can import cURL examples. [16]

5) Add a response strategy. Option A: In Webhook node set "Respond: When Last Node Finishes."
Option B: Use **Respond to Webhook** node for controlled responses. n8n documents that Respond to Webhook runs once for the first data item and defines workflow behavior for missing respond node, errors before respond node, and multiple respond nodes. [63]

**Testing**

- Send a well-formed JSON payload and verify success response.
- Send a payload missing `email` and verify error behavior and logs.

**Reliability extensions (instructor choice)**
Discuss max payload (16MB) and how it can be configured in self-hosted mode (N8N_PAYLOAD_SIZE_MAX). [18]

**Troubleshooting quick checks** - Are you hitting the **Test URL** while the workflow is listening? [18]
- Did you publish/activate before using the **Production URL**? [18]
- If behind reverse proxy, did you set WEBHOOK_URL so n8n displays/registers correct URLs? [19]

**Screenshot request (official docs)** - S1: Webhook node panel showing Test vs Production URL and Respond options. [18]

**Lab n8n B: ETL sync with rate-limit controls**

**Level:** Intermediate
**Archetype:** Scheduled run → API fetch → transform → batch upserts → monitor

**Objective:** Build a scheduled workflow that fetches data from an API, transforms it into a stable schema, and writes it to a destination (spreadsheet/DB/API). The lab's focus is **rate limits** and resilient API patterns.

**Key documentation anchor:** n8n's HTTP Request node supports REST calls, cURL import, and provides built-in mitigation suggestions for HTTP 429 via batching and retries. [64]

**Workflow overview diagram**

```
flowchart LR
  S[Schedule trigger<br/>every N minutes] --> F[HTTP GET /items]
  F --> P[Parse + map fields<br/>normalize schema]
  P --> B[Batching controls<br/>items per batch + delay]
  B --> U[HTTP POST/PUT upsert]
  U --> L[Log result summary]
```

**Step-by-step build**

1) Trigger: use a schedule/cron trigger appropriate for your class environment.

2) Fetch: add **HTTP Request** node - Use GET - Demonstrate "Import cURL" by pasting a cURL example from an API doc. n8n calls out the import feature explicitly. [16]

3) Normalize: use expressions or Code node to transform data into a clean internal schema. Reinforce that n8n internal items should be shaped as `{ json: ... }`. [15]

4) Handle rate limits: - Configure **Batching** and/or enable **Retry on Fail**, as recommended for 429 responses. [23]

5) Write: use HTTP Request to send updates to destination service; use PUT where possible for idempotency to reduce duplicate creation risk. [14]

**Testing**

- Run once with a small dataset.
- Simulate a 429 scenario (or explain how to trigger it) and show adjustments via Batching or Retry.

**Troubleshooting playbook** - 400 “bad request”: check query params and JSON formatting. [23]
- 403 “forbidden”: credential permissions/scopes. [23]
- 429: reduce request frequency; batch; retry with wait. [23]

**Lab n8n C: Production-style orchestration with error workflow**

**Level:** Advanced
**Archetype:** Webhook intake → branch → side effects → error workflow → queue-mode readiness

**Objective:** Implement a workflow that: - processes a webhook event, - performs multiple dependent actions, - uses an **Error Trigger**-based error workflow for centralized error handling, - and includes deployment notes for scaling/queue mode.

**Why this lab matters:** n8n’s Error Trigger is explicitly limited: it only runs when an **automatic** workflow errors, and cannot be tested by manual runs. [65] This is a concrete example of “production ≠ manual test” behavior.

**Core workflow diagram**

```
flowchart TB
  WH[Webhook trigger] --> NORM[Normalize data]
  NORM --> ROUTE{Route by type}
  ROUTE -->|type=A| A[Action chain A]
  ROUTE -->|type=B| B[Action chain B]
  A --> DONE[Success log]
  B --> DONE
```

**Error workflow diagram**

```
flowchart LR
  ET[Error Trigger] --> EX[Extract error details]
  EX --> NOTIF[Notify: chat/email/ticket]
  NOTIF --> STORE[Store error record]
```

**Step-by-step build**

1) Build the main workflow with a Webhook trigger and at least two branches.

2) Create a separate error workflow containing **Error Trigger** and steps to notify/store error info.

3) Register the error workflow: - Set the error workflow in workflow settings (UI-dependent). - Verify behavior with a production execution (not manual). n8n documentation states you cannot test it manually because Error Trigger only runs on automatic workflow errors. [65]

4) Deployment notes for queue mode: - If you scale into queue mode, ensure Postgres + Redis connectivity and version alignment across mains/workers; sticky sessions behind load balancer for multi-main. [10] - Ensure the same credential encryption key is available to all workers in queue mode. [24]

**Troubleshooting** - If error workflow fails to run during manual testing: expected behavior. [65]
- If webhook works in test but not production: confirm publish + production URL semantics. [18]
- If behind a reverse proxy: configure WEBHOOK_URL and proxy hops. [19]

## Labs for Zapier

**Lab Zapier A: Two-app automation with clear task accounting**

**Level:** Beginner
**Archetype:** Trigger → action (two-step Zap)

**Objective:** Build a basic Zap that triggers on an event (new item in app A) and performs a single action (create/post in app B). Measure tasks consumed and interpret Zap history.

**Key facts to teach** - Trigger doesn't count as a task. - Each successful action counts as a task. [53] - Zap history is your runtime log; retention is limited (60 days guaranteed, up to 10,000 runs displayed). [21]

**Workflow diagram**

```
flowchart LR
  T[Trigger] --> A[Action]
  A --> H[Zap history record]
```

**Step-by-step build**

1) Create Zap: select a trigger app/event and test it.

2) Add one action: choose an action app/event and test it.

3) Turn on Zap and generate 3 sample events.

4) Open **Zap history** and observe: - run status - data in/out step - task usage view [21]

**Task accounting exercise** - If 3 events triggered and the action succeeded 3 times, you used 3 tasks (trigger checks do not count). [53]

**Lab Zapier B: Data cleaning + gating using Formatter and Filters**

**Level:** Intermediate
**Archetype:** Trigger → Formatter → Filter → action(s)

**Objective:** Enforce data quality before side effects: - Normalize user input (date, email casing, numbers). - Stop the Zap if conditions fail (e.g., missing required fields or wrong product type).

Zapier documentation explains: - Formatter transforms text/numbers/dates and can draft steps with "Formatter with AI." [66] - Filters allow Zaps to proceed only when criteria are met; you add filter steps after the trigger. [20]

**Workflow diagram**

```
flowchart LR
  T[Trigger<br/>new record] --> FMT[Formatter<br/>normalize fields]
  FMT --> FIL{Filter<br/>meets condition?}
  FIL -->|yes| ACT[Action<br/>create/update]
  FIL -->|no| STOP[Stop (no actions)]
```

**Step-by-step build**

1) Build trigger and test.

2) Add Formatter step to: - reformat date/time, numbers, or text. [66]

3) Add Filter step: - choose a field to check, rule, and expected value. Zapier's Filter instructions (add step, select Filter, configure rules) are explicit. [20]

4) Add action step only after filter passes.

**Cost insight** - Built-in tools like Filter and Formatter do not count as tasks, per Zapier's pricing page listing of non-task steps. [53]

**Troubleshooting** - If filter "always runs" warning appears: you mapped the same field as both condition and compare value (a tautology). [20]

**Lab Zapier C: Advanced branching with Paths + Code step + operational guardrails**

**Level:** Advanced
**Archetype:** Trigger → Paths branches → per-branch actions → careful run history + error handling behavior

**Objective:** Build a multi-branch Zap that routes based on conditions, uses a code step for a non-trivial transform, and respects platform limits.

**Constraints to highlight** - Zaps limited to 100 steps (including steps inside paths). [67] - Paths: up to 10 branches per group, up to 3 nested path steps, and Paths must be final step in Zap. [22]

**Workflow diagram**

```
flowchart TB
  T[Trigger] --> C[Code step<br/>compute classification]
  C --> P{Paths<br/>route by class}
  P -->|A| A1[Actions A]
  P -->|B| B1[Actions B]
  P -->|C| C1[Actions C]
```

**Step-by-step build**

1) Choose a trigger with a payload containing text (e.g., form submission, email subject, ticket title).

2) Add **Code by Zapier** step (Run Javascript). Zapier's docs specify: - Node.js 18 environment - uses `inputData` - UTC timezone in Code actions [17]

Example:

```
// Code by Zapier (JavaScript, Node.js 18)
const text = (inputData.text || "").toLowerCase();

let category = "general";
if (text.includes("refund") || text.includes("invoice")) category = "billing";
if (text.includes("error") || text.includes("bug")) category = "technical";

return { category };
```

3) Introduce Paths: - Add Paths step (creates Path A and Path B by default). - Add rules per path (e.g., category equals "billing" vs "technical"). Zapier documents the maximal nesting and step limits and notes Paths must be final step. [60]

4) Add actions inside each path (e.g., tag record, notify different channel, create a ticket).

5) Turn on and test with 6–10 events across categories.

**Operational guardrails** - Check Zap history for run failures and step-level data. - Emphasize that Zaps can be automatically turned off if error ratio is too high (95% in last 7 days). [39]

### Speaker notes

Labs are where students learn that “automation” is primarily “interfaces + failure modes.” For each lab, enforce three questions: - What is the input contract? - What are the side effects? - How do we prevent duplicates and diagnose failures?

Use Zapier history and n8n executions as evidence sources, showing students where to look before they ask for help. [68]

## Comparative analysis

### Capability and ecosystem comparison

| Dimension | n8n | Zapier | Analytical takeaway |
|---|---|---|---|
| Integration ecosystem | “400+” in repo; “500+” in marketing; plus HTTP for any API [69] | 8,000+ apps/ integrations [4] | Zapier wins breadth; n8n’s generic HTTP + code reduces long-tail gaps |
| Hosting | Cloud + self-host; self-hosting recommended for experts [70] | Managed SaaS; enterprise options like VPC peering are marketed in plans [71] | Choose based on operational maturity and data residency needs |
| Pricing primitive | Workflow executions per month [5] | Tasks per successful action; built-in tools may not count [6] | n8n favors complex workflows; Zapier cost scales with external actions |
| Scaling pattern | Queue mode with Redis + Postgres + workers [10] | Managed scaling; “horizontal scalability” messaging [72] | n8n gives knobs; Zapier abstracts knobs |
| Limits | Depends on hosting/ resources; queue mode adds complexity [73] | 100-step cap; path nesting caps; step field caps [60] | Zapier forces modular decomposition; n8n pushes you toward ops skills |
| Observability | Execution logging, audit logging described in product pages [74] | Zap history with 60-day guarantee; audit log available [30] | Both support debugging; retention/exports differ |

| Dimension | n8n | Zapier | Analytical takeaway |
|---|---|---|---|
| Extensibility | Code node + community nodes; community nodes require self-hosting for npm installs [75] | Webhooks + Code steps; code runs in constrained Node.js env [76] | n8n is more “platform-like”; Zapier is more “workflow-as-a-service” |

## Cost modeling: executions vs tasks

**n8n:** One execution = one full workflow run regardless of step count and “how much data it processes” (per their pricing explanation). [47] This pushes design toward: - richer workflows that do more per run - batching multiple items per execution when possible

**Zapier:** Each successful action counts as a task; triggers and many built-in steps do not count; only success counts. [6] This pushes design toward: - using Filters/Formatter/Paths “freely” - minimizing expensive external action steps (and avoiding replay/duplicate success)

Zapier also describes pay-per-task billing when exceeding plan limits (switching to pay-per-task at 1.25x base task cost, with additional cap logic). [53]

## Security and governance comparison

- Zapier emphasizes managed compliance (SOC 2 Type II & SOC 3 certifications) and encryption (TLS, AES-256 at rest), plus identity controls (SSO/SCIM/2FA) in enterprise materials. [77]
- n8n Cloud describes encryption at rest (Azure SSE, AES-256 FIPS 140-2 implementation), TLS in transit, and audit log retention (at least 12 months history, per policy description). [45]
- n8n self-hosting shifts those responsibilities to you, and n8n explicitly warns self-hosting mistakes can cause security issues and downtime. [78]

## Decision framework for selecting a platform

Use constraints-first selection:

- If you need **fast adoption by non-technical stakeholders**, broad integration coverage, and minimal platform operations: Zapier is typically the default. [79]
- If you need **deployment control** (self-hosting, specialized networking), more explicit engineering hooks (worker scaling), or want to build complex workflows without per-step pricing: n8n is typically the better fit—assuming operational competency. [80]

## Speaker notes

Tell students that “best tool” is context-dependent. Use a decision memo exercise: each group chooses a platform for a scenario and must cite constraints (step caps, compliance needs, self-hosting warning, audit requirements, cost primitive). [81]

# Best practices, troubleshooting, use cases, ethics, and future trends

## Best practices that generalize across both platforms

**Design** - Separate workflows into layers: intake/normalize → decision → side effects → reporting. - Favor explicit schemas: define “required fields” and normalize early. - Use idempotency keys for side effects to prevent duplicates when retries/replays occur. [82]

**Testing** - Test with representative payloads (including edge cases: missing fields, nulls, large arrays). - For webhook-driven flows: verify both test and production modes. - In n8n, test vs production behavior is explicitly different. [18] - In Zapier webhooks, validate that the correct trigger URL is used and understand raw vs parsed body and size limits. [56]

**Deployment** - For n8n behind reverse proxies: set WEBHOOK_URL and proxy headers/hops to ensure correct generated webhook URLs. [19] - For n8n queue mode: keep encryption key consistent across workers and ensure Redis/Postgres prerequisites. [83]

**Observability** - Zapier: teach students to treat Zap history as the first stop; it includes run logs, version info, and retention constraints. [21] - n8n: teach students to treat execution logs and error workflows as observability primitives. [84]

## Troubleshooting scenarios and playbooks

**Scenario: “Webhook works in test but not in production”**

- **n8n**: verify that production webhook is registered only when workflow is published/active; production runs don’t display data in the editor the same way, but are visible via executions. [18]
- **Zapier**: verify that the webhook URL copied from the trigger step is current; choose Catch Hook vs Catch Raw Hook based on whether you need parsed vs raw data and confirm size limits. [56]

**Scenario: “We are getting rate limit errors”**

- **n8n**: apply batching and retry-on-fail as recommended for 429 responses. [23]
- **Zapier**: webhooks can be throttled under heavy concurrent load (Zapier notes delays and points to webhook rate limits). [56]

**Scenario: “The automation turned itself off”**

Zapier’s error-ratio policy: Zaps automatically turn off when 95% of runs error in the last 7 days. [39] Treat this as an operational alarm: - Inspect Zap history to diagnose repeating failure patterns. - Fix root cause before replaying runs (since replay can re-consume tasks and repeat actions). [40]

**Scenario: “Credentials suddenly can’t be decrypted” (n8n self-host / scaled)**

n8n credentials depend on an encryption key generated at first launch or configured via N8N_ENCRYPTION_KEY; in queue mode, all workers must share it. [24] This scenario teaches: - configuration drift is real - keys are not “set-and-forget”

## Real-world use cases

**Education use case: automated assignment triage**

- Intake: form submission/webhook
- Normalize: ensure student ID, course code, deadline
- Route: urgent deadlines → notify TA channel; regular → append to tracker
- Audit: store run logs and export periodically (Zapier retention 60 days). [21]

**Research use case: literature ingestion pipeline**

- Trigger: scheduled job
- Fetch: call APIs (papers/metadata)
- Transform: extract authors, DOI, keywords
- Store: push to database/spreadsheet
- Rate-limits: apply batching/retry patterns (n8n built-in suggestions). [23]

**Small business use case: lead routing and follow-up**

- Trigger: new lead from form or CRM
- Clean: formatter normalize phone and email (Zapier)
- Gate: filter out spam or missing required info
- Branch: enterprise value leads vs small leads (Paths)
- Notify: sales channel + create task
- Cost: minimize external actions; built-in tools not counted as tasks. [85]

## Ethical and security considerations

Automation platforms routinely process personal data (emails, phone numbers, IDs). Ethical design means preventing automation from becoming “silent surveillance” or a data exfiltration channel.

**Security-of-processing as a principle:** GDPR’s Article 32 frames security measures as risk-based and explicitly includes encryption as an example measure. [25] Even if you are not legally bound by GDPR, the design thinking is a strong baseline.

**Privacy and cybersecurity frameworks:** The National Institute of Standards and Technology [86] CSF 2.0 structures cybersecurity outcomes into Govern, Identify, Protect, Detect, Respond, Recover, encouraging lifecycle risk management. [87] The NIST Privacy Framework complements this with Identify-P, Govern-P, Control-P, Communicate-P, Protect-P to manage privacy risks from data processing. [88]

**API security risks:** Automation often stitches together APIs; the OWASP [89] API Security Top 10 lists common failures such as Broken Object Level Authorization (BOLA), reinforcing least privilege and authorization checks when accessing objects by ID. [90]

**Minimal ethical checklist for student projects** - Collect only what you need (data minimization). - Document where data goes (data flow diagram, including third-party services). - Use least-privilege tokens and separate credentials per workflow. - Prefer idempotent operations to reduce duplicate harm. - Ensure auditability: store run records and export logs when platform retention is limited. [30]

### Future trends

Three trends are shaping automation tools: - **AI-assisted building and runtime augmentation**: Both platforms highlight AI-related capabilities (e.g., "AI Workflow Builder credits" in n8n plans; AI fields and AI orchestration language in Zapier plans). [91] - **Agentic tool use**: Zapier's task usage documentation references tool calls in "Zapier MCP" counting as tasks, indicating that automation is expanding from deterministic workflows into tool-assisted agent execution with metered usage. [92] - **Governance as product surface**: Audit logs, app controls, and data retention are increasingly explicit features (Zapier enterprise materials; n8n audit logging and log streaming mentioned in plan comparison). [93]

### Speaker notes

End this section by reframing automation as "software supply chain for business processes." Students should leave with a security mindset: every connector is an integration boundary and a potential data leak. Tie OWASP API risks directly to workflows that accept arbitrary webhook payloads. [94]

## Seminar assets and production plan

### Slide outline with speaker notes

**Slide: Why automation** - Key message: automation reduces repetitive work and error rates, but introduces reliability and governance responsibilities. - Speaker notes: Start with a concrete story: "a missed email → missed deadline." Then show that automation is a control system, not a convenience toy.

**Slide: Workflow definition** - Quote/paraphrase WFMC workflow definition. - Speaker notes: Emphasize "procedural rules" and "passing information/tasks," not "drag-and-drop."

**Slide: Shared mental model** - Show the automation pipeline diagram (trigger → validate → rules → actions → logs). - Speaker notes: Students should memorize this as their debugging roadmap.

**Slide: Cost primitives** - n8n executions vs Zapier tasks. - Speaker notes: One execution covers the whole workflow; tasks count successful actions; built-in tools may not count. [11]

**Slide: n8n deep dive** - Architecture and queue mode. - Speaker notes: Teach why Redis/Postgres appear and what "sticky sessions" imply. [10]

**Slide: Zapier deep dive** - Zaps, step limits, Paths constraints. - Speaker notes: Explain 100-step cap as an architectural forcing function. [95]

**Slide: Lab summary** - Six labs and what each teaches. - Speaker notes: Pair each lab to a professional archetype (notify / ETL / orchestration).

**Slide: Observability** - Zap history and retention; n8n execution logs and error workflows. - Speaker notes: “Logs are part of the product.”

**Slide: Troubleshooting with playbooks** - High-frequency failure modes: auth errors, rate limits, webhook confusion, data mapping errors. - Speaker notes: Make students role-play SRE: “what evidence do you collect first?”

**Slide: Security and ethics** - NIST CSF functions + NIST Privacy functions + OWASP API risks. - Speaker notes: Connect to “automation is API glue,” so API security is automation security. [8]

**Slide: Capstone prompt** - “Design an automation for X under constraints Y.” - Speaker notes: Grade on correctness + reliability + governance, not on number of steps.

## One-page cheat sheet

**Core concepts** - Trigger: the event that starts a workflow (Zapier trigger; n8n trigger node). [96]
- Action/Step: work performed after trigger.
- n8n execution: one full run of a workflow; step count doesn’t change the count. [47]
- Zapier task: each successful action; triggers don’t count; many built-in tools don’t count. [6]

**Platform quick picks** - Choose Zapier if: you need broad app coverage (8,000+), managed ops, rapid adoption. [4]
- Choose n8n if: you need deployment control, queue-mode scaling knobs, complex workflows per run. [97]

**Do-this-first debugging** - Zapier: open Zap history → locate failed run → inspect step data → fix mapping/credentials → replay carefully. [98]
- n8n: open execution view/logs → inspect node output → confirm webhook URL mode (test vs prod) → validate credentials and encryption key consistency in scaled setups. [99]

**Common limits** - Zapier: 100 steps total; Paths up to 10 branches; 3 nested; Paths must be last. [60]
- n8n Webhook payload: 16MB default. [18]

**Security checklist** - Least privilege tokens; isolate credentials per workflow; protect webhooks (auth/IP allowlist where possible); encrypt data in transit; export logs if retention limited. [100]

## Seven-day production timeline

**Day 1: Research + architecture scaffolding** - Validate pricing/limits/security claims against official pages for both platforms. - Draft figure/table list and define screenshot capture list (official docs). - Lock page plan and section word budgets. [101]

**Day 2: Foundations writing** - Workflow definition, patterns, idempotency, error handling. - Draft Figures 1–3 and Tables 1–2. [102]

**Day 3: n8n deep dive + labs A/B** - n8n overview, hosting warning, webhook semantics, HTTP Request patterns. - Create mermaid diagrams and code listings; add screenshot placeholders. [103]

**Day 4: n8n lab C + scaling/security** - Queue mode architecture, encryption key guidance, error workflow semantics. [104]

**Day 5: Zapier deep dive + labs A/B** - Task accounting, built-in tool cost behavior, filters/formatter, Zap history. [105]

**Day 6: Zapier lab C + comparison** - Paths constraints, code step environment, troubleshooting and run governance. - Write comparison matrix and decision rubric. [106]

**Day 7: Edit for print + speaker notes + final QA** - Ensure consistent terminology; verify citations; finalize TOC/list of figures/tables/code. - Add one-page cheat sheet and slide outline; run a final “scenario walkthrough” proofread.

## Speaker notes

Tell students that “production readiness” is the difference between a cool demo and a reliable system. Use the timeline to show professional document assembly: research → draft → labs → compare → secure → polish.

---


[1] [18] [29] [33] [44] [86] [89] [99] [100] https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.webhook/
https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.webhook/

[2] [102] https://www.aiai.ed.ac.uk/project/wfmc/ARCHIVE/DOCS/glossary/glossary.html
https://www.aiai.ed.ac.uk/project/wfmc/ARCHIVE/DOCS/glossary/glossary.html

[3] [42] https://n8n.io/
https://n8n.io/

[4] https://zapier.com/
https://zapier.com/

[5] [11] [41] [46] [47] [48] [49] [74] [84] [91] [101] https://n8n.io/pricing/
https://n8n.io/pricing/

[6] [34] [53] [71] [85] [96] [105] https://zapier.com/pricing
https://zapier.com/pricing

[7] https://zapier.com/blog/n8n-vs-zapier/
https://zapier.com/blog/n8n-vs-zapier/

[8] [87] https://nvlpubs.nist.gov/nistpubs/CSWP/NIST.CSWP.29.pdf
https://nvlpubs.nist.gov/nistpubs/CSWP/NIST.CSWP.29.pdf

9 21 30 36 68 98 https://help.zapier.com/hc/en-us/articles/8496291148685-View-and-manage-your-Zap-history
https://help.zapier.com/hc/en-us/articles/8496291148685-View-and-manage-your-Zap-history

10 27 51 73 97 104 https://docs.n8n.io/hosting/scaling/queue-mode/
https://docs.n8n.io/hosting/scaling/queue-mode/

12 22 35 60 106 https://help.zapier.com/hc/en-us/articles/8496288555917-Add-branching-logic-to-Zaps-with-Paths
https://help.zapier.com/hc/en-us/articles/8496288555917-Add-branching-logic-to-Zaps-with-Paths

13 45 https://n8n.io/legal/security/
https://n8n.io/legal/security/

14 82 https://www.rfc-editor.org/rfc/rfc9110.html
https://www.rfc-editor.org/rfc/rfc9110.html

15 62 https://docs.n8n.io/courses/level-two/chapter-1/
https://docs.n8n.io/courses/level-two/chapter-1/

16 64 https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.httprequest/
https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.httprequest/

17 https://help.zapier.com/hc/en-us/articles/8496310939021-Use-JavaScript-code-in-Zaps
https://help.zapier.com/hc/en-us/articles/8496310939021-Use-JavaScript-code-in-Zaps

19 https://docs.n8n.io/hosting/configuration/configuration-examples/webhook-url/
https://docs.n8n.io/hosting/configuration/configuration-examples/webhook-url/

20 32 https://help.zapier.com/hc/en-us/articles/8496276332557-Add-conditions-to-Zaps-with-filters
https://help.zapier.com/hc/en-us/articles/8496276332557-Add-conditions-to-Zaps-with-filters

23 28 https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.httprequest/common-issues/
https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.httprequest/common-issues/

24 83 https://docs.n8n.io/hosting/configuration/configuration-examples/encryption-key/
https://docs.n8n.io/hosting/configuration/configuration-examples/encryption-key/

25 https://eur-lex.europa.eu/legal-content/EN/TXT/PDF/?uri=CELEX%3A32016R0679
https://eur-lex.europa.eu/legal-content/EN/TXT/PDF/?uri=CELEX%3A32016R0679

26 90 94 https://owasp.org/API-Security/editions/2023/en/0x11-t10/
https://owasp.org/API-Security/editions/2023/en/0x11-t10/

31 https://www.vdaalst.com/publications/p108.pdf
https://www.vdaalst.com/publications/p108.pdf

37 66 https://help.zapier.com/hc/en-us/articles/8496212590093-Get-started-with-Formatter
https://help.zapier.com/hc/en-us/articles/8496212590093-Get-started-with-Formatter

38 https://www.vdaalst.com/publications/p325.pdf
https://www.vdaalst.com/publications/p325.pdf

39 https://help.zapier.com/hc/en-us/articles/8496037690637-How-to-troubleshoot-errors-in-Zaps
https://help.zapier.com/hc/en-us/articles/8496037690637-How-to-troubleshoot-errors-in-Zaps

40 54 92 https://help.zapier.com/hc/en-us/articles/8496196837261-How-is-task-usage-measured-in-Zapier
https://help.zapier.com/hc/en-us/articles/8496196837261-How-is-task-usage-measured-in-Zapier

43 61 70 78 80 81 103 https://docs.n8n.io/hosting/
https://docs.n8n.io/hosting/

50 https://docs.n8n.io/sustainable-use-license/
https://docs.n8n.io/sustainable-use-license/

52 79 https://help.zapier.com/hc/en-us/articles/21996626006541-Introduction-to-apps-on-Zapier
https://help.zapier.com/hc/en-us/articles/21996626006541-Introduction-to-apps-on-Zapier

55 67 95 https://help.zapier.com/hc/en-us/articles/8496181445261-Zap-limits
https://help.zapier.com/hc/en-us/articles/8496181445261-Zap-limits

56 76 https://help.zapier.com/hc/en-us/articles/8496288690317-Trigger-Zaps-from-webhooks
https://help.zapier.com/hc/en-us/articles/8496288690317-Trigger-Zaps-from-webhooks

57 72 77 https://zapier.com/security-compliance
https://zapier.com/security-compliance

58 https://help.zapier.com/hc/en-us/articles/8496181993613-Security-and-Compliance
https://help.zapier.com/hc/en-us/articles/8496181993613-Security-and-Compliance

59 https://help.zapier.com/hc/en-us/articles/13295074298125-Use-the-audit-log-to-review-your-account-activity
https://help.zapier.com/hc/en-us/articles/13295074298125-Use-the-audit-log-to-review-your-account-activity

63 https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.respondtowebhook/
https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.respondtowebhook/

65 https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.errortrigger/
https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.errortrigger/

69 https://github.com/n8n-io/n8n
https://github.com/n8n-io/n8n

75 https://docs.n8n.io/code/code-node/
https://docs.n8n.io/code/code-node/

88 https://nvlpubs.nist.gov/nistpubs/CSWP/NIST.CSWP.01162020.pdf
https://nvlpubs.nist.gov/nistpubs/CSWP/NIST.CSWP.01162020.pdf

93 https://pagebuilder.vercel.zapier.com/security-compliance
https://pagebuilder.vercel.zapier.com/security-compliance